JOBSHEET IX

Mata Kuliah: Praktikum Pemrograman Berbasis Objek

Abstrak dan Interface

Dosen Pengampu:

Dian Hanifudin Subhi, S.Kom., M.Kom.



Disusun oleh:

Ahmad Rafid Riqkullah

254107020078

T1 – 2B

D-IV Teknik Informatika

Tanggal: 23 Oktober 2026

POLITEKNIK NEGERI MALANG

Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur 65141

TAHUN 2026-2027

C. Langkah Kerja

Langkah 1: Account Menjadi Kelas Abstrak

Code:

```
57
58      // Method abstrak yang wajib di-override oleh seluruh subclass
59      public abstract double monthlyFee();
60  }
```

Figure 1 Account.java menjadi abstract class dengan method abstrak monthlyFee

```
36      // Tambahan method baru untuk Langkah 1
37      public void printMonthlyFees() {
38          for (int i = 0; i < count; i++) {
39              System.out.println(accounts[i].getAccountNumber() + " fee: " + accounts[i].monthlyFee());
40          }
41      }
42  }
```

Figure 2 Bank.java dengan method printMonthlyFees

```
31      // Implementasi method abstrak dari Account
32      @Override
33      public double monthlyFee() {
34          return 0;
35      }
36  }
```

Figure 3 Bank.java dengan method printMonthlyFees

```
31      // Implementasi method abstrak dari Account
32      @Override
33      public double monthlyFee() {
34          return MONTHLY_FEE;
35      }
36  }
```

Figure 4 CheckingAccount.java mengisi monthlyFee

Main.java ...\Js9 M | Main.java (5108931) | CheckingAccount.java ...\Js9 | InterestBearing.java U | SavingsAcco

C: > Users > IdeaPad > Documents > SEMESTER 3 > Praktikum Program Berbasis Objek > Praktikum-Pemrograman-Berbasi-Object-PBO- > Jobsheet9

Pidddd, 2 days ago | author (Pidddd)

```
1   package id.ac.polinema.Js9;        Pidddd, 2 days ago • Langkah 1
2
3   public class Main {
4       public static void main(String[] args) {
5           Customer customer1 = new Customer("Nadia", "0812-0000-0001");
6           SavingsAccount acc1 = new SavingsAccount("A001", customer1, 500000, 0.01);
7           acc1.withdraw(150000);
8
9           Customer customer2 = new Customer("Sari", "0812-0000-0002");
10          CheckingAccount acc2 = new CheckingAccount("A002", customer2, 200000, 50000);
11
12          Customer customer3 = new Customer("Rian", "0812-0000-0003");
13          SavingsAccount savings = new SavingsAccount("A003", customer3, 100000, 0.02);
14          savings.withdraw(70000);
15
16          Customer customer4 = new Customer("Dewi", "0812-0000-0004");
17          CheckingAccount checking = new CheckingAccount("A004", customer4, 100000, 200000);
18          checking.withdraw(250000);
19
20          Bank bank = new Bank(10);
21          bank.addAccount(acc1);
22          bank.addAccount(acc2);
23          bank.addAccount(savings);
24          bank.addAccount(checking);
25
26          bank.printAllAccounts();
27          System.out.println("");
28          bank.printMonthlyFees();
29      }
30  }
```

Figure 5 Main.java memanggil printMonthlyFees

Hasil:

```
PS C:\Users\IdeaPad\Documents\SEMESTER 3\Praktikum Program Berbasis Obj
2-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInEx
raktikum-Pemrograman-Berbasi-Object-PBO-_2f5b4ee8\bin' 'id.ac.polinema.
A001 Nadia balance: 350000.0
Account type: Savings, interest rate: 0.01
A002 Sari balance: 200000.0
Account type: Checking, overdraft limit: 50000.0
A003 Rian balance: 100000.0
Account type: Savings, interest rate: 0.02
A004 Dewi balance: -150000.0
Account type: Checking, overdraft limit: 200000.0

A001 fee: 0.0
A002 fee: 15000.0
A003 fee: 0.0
A004 fee: 15000.0
PS C:\Users\IdeaPad\Documents\SEMESTER 3\Praktikum Program Berbasis Obj
```

Figure 6 Hasil Percobaan Langkah 1

Penjelasan: Pada langkah ini, Account diubah menjadi kelas abstrak agar tidak bisa dibuat menjadi objek polos melalui perintah new. Deklarasi method abstrak monthlyFee() di kelas Account mewajibkan setiap subclass untuk menentukan biaya bulanannya sendiri. Saat method bank.printMonthlyFees() dijalankan, program memanfaatkan polimorfisme untuk memanggil monthlyFee() milik kelas aslinya, sehingga rekening tabungan menghasilkan biaya 0.0 dan rekening giro menghasilkan biaya 15000.0.

Langkah 2: InterestBearing, Interface untuk Rekening Berbunga

Code:

ount.java ...\polinema U CheckingAccount.java ...\poline

Jobsheet9 > id > ac > polinema > InterestBearing.java >

```java
package id.ac.polinema;

public interface InterestBearing {
    void applyInterest();
}

```

Figure 7 InterestBearing.java

```java
    // Implementasi method dari interface InterestBearing
    @Override
    public void applyInterest() {
        double interest = getBalance() * interestRate;
        deposit(interest);
    }
}
```

Figure 8 SavingsAccount.java meng-implement InterestBearing

```
        // Tambahan uji Langkah 2
        System.out.println("Before interest: " + savings.getBalance());
        savings.applyInterest();
        System.out.println("After interest: " + savings.getBalance());
    }
}
```

Figure 9 Main.java menguji applyInterest

Hasil:

```
Before interest: 100000.0
After interest: 102000.0
PS C:\Users\IdeaPad\Documents\SEMESTER
SmpPgriJabungMalang#3 needs reviewers  ⊗ 15 ⚠ 0
```

Figure 10 Hasil Percobaan Langkah 2

Penjelasan: Interface InterestBearing dibuat untuk memberikan kemampuan menerima bunga secara khusus. Hanya kelas SavingsAccount yang menyematkan implements InterestBearing, sehingga rekening giro tidak terbebani oleh fitur bunga. Saat method applyInterest() dipanggil pada rekening tabungan A003, sistem menghitung bunga sebesar 2% dari saldo 100.000 (2.000), lalu menyetorkannya otomatis hingga saldo akhir bertambah menjadi 102.000.

D. Tugas dan Hasil Kerja

- Screenshot output program setelah Langkah 2.

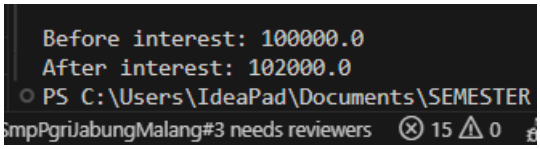


Figure 11 Hasil Percobaan Langkah 2

- Tugas mandiri:

1. Bank memerlukan jejak audit untuk rekening yang saldonya bisa negatif (rekening dengan overdraft). Tambahkan interface Auditable, lalu terapkan pada CheckingAccount. Diagram berikut hanya sketsa method yang perlu diisi, BUKAN kode jadi, isi auditLog() diserahkan sepenuhnya padamu:

   Buktikan lewat Main.java: panggil auditLog() pada kedua CheckingAccount yang sudah ada, lalu cetak hasilnya. Hasilmu benar apabila:

   - Auditable adalah interface dengan satu method auditLog();
   - CheckingAccount menyatakan implements Auditable, sedangkan SavingsAccount tidak;
   - teks yang dicetak memuat nomor rekening dan saldonya.

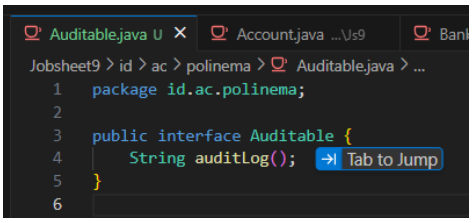


Figure 12 Class Interface Auditable

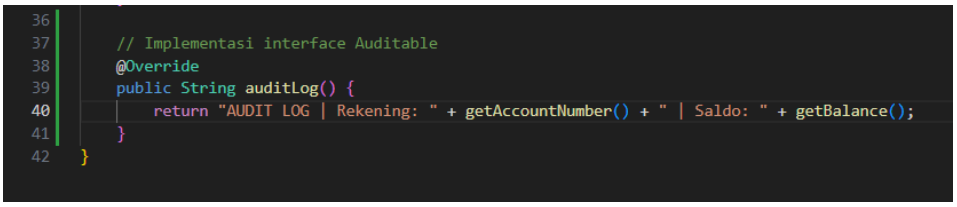


Figure 13 Class CheckingAccount Auditable

Hasil:

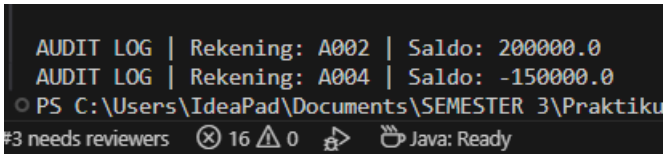


Figure 14 Hasil Latihan Mandiri

Penjelasan: Interface Auditable dibuat dengan method auditLog() untuk memenuhi kebutuhan jejak audit pada rekening dengan fasilitas *overdraft* (penarikan saldo sampai minus). Interface ini hanya diimplementasikan pada CheckingAccount tanpa mengubah kelas SavingsAccount. Output membuktikan bahwa pemanggilan auditLog() pada kedua akun giro berhasil mencetak nomor rekening beserta saldonya, baik yang bernilai positif (200.000) maupun yang bersaldo negatif (-150.000).

2. Jawab secara singkat (1-2 kalimat untuk masing-masing pertanyaan):

   (a) Mengapa applyInterest() dijadikan interface InterestBearing, bukan method abstrak di Account?

   Jawab: Karena bunga hanya berlaku untuk rekening tabungan, sedangkan rekening giro tidak berbunga. Jika ditaruh di Account, rekening giro akan dipaksa mengisinya padahal tidak membutuhkan fitur tersebut.

   (b) SavingsAccount memakai extends Account dan implements InterestBearing. Apa beda makna kedua kata kunci itu?

   Jawab: extends menunjukkan identitas keluarga inti (*is-a*), yaitu SavingsAccount adalah turunan dari Account dan mewarisi data dasarnya. Sedangkan implements menunjukkan kemampuan tambahan (*can-do*), yaitu memberi kontrak agar kelas tersebut bisa menjalankan fitur hitung bunga.

Hasil dan Pembahasan

Program Bank Mini berhasil mengimplementasikan konsep kelas abstrak dan interface menggunakan bahasa pemrograman Java. Kelas Account dijadikan kelas abstrak untuk mencegah pembuatan rekening umum sekaligus menetapkan kontrak method abstrak monthlyFee() yang wajib diisi oleh SavingsAccount dan CheckingAccount sesuai aturan biayanya. Pemisahan fungsionalitas dilakukan melalui interface InterestBearing khusus untuk rekening tabungan yang berbunga, serta interface Auditable khusus untuk rekening giro yang memerlukan pencatatan log saldo dan limit *overdraft*. Mekanisme ini memastikan arsitektur kode tetap terstruktur, rapi, dan modular.

Kesimpulan

Kelas abstrak berfungsi sebagai kerangka dasar (blueprint) untuk menyatukan atribut dan perilaku umum yang tidak boleh diinstansiasi secara langsung. Sementara itu, interface berperan sebagai kontrak kemampuan (can-do) yang fleksibel untuk menambahkan fitur spesifik ke kelas tertentu tanpa terikat batasan pewarisan tunggal (single inheritance). Kombinasi keduanya memungkinkan pembuatan sistem berorientasi objek yang fleksibel dan menerapkan pemisahan tugas secara tegas.

Lampiran

Link Github Ahmad Rafid Riqkullah atau https://github.com/Pidddd/Praktikum-Pemrograman-Berbasi-Object-PBO-/tree/main/Jobsheet9/id/ac/polinema